ChatGPT

줌줌투어 전사 온톨로지·지식그래프 구축 심층 조사

핵심 결론 한 줄: 1차 구축은 **Neo4j Community + Git을 원본(Source of Truth)으로 하는 메타데이터 저장소 + 공식 Neo4j MCP의 읽기 전용 모드 + 가벼운 DB 스키마 수집기** 조합을 가장 권장합니다. Neo4j Community는 GPLv3 이지만 사내·폐쇄망에서 상업적으로 무료 사용이 가능하고, 최신 공식 MCP에는 쓰기 도구를 제거하는 `NEO4J_READ_ONLY=true`가 있습니다. [1]
핵심 결론 두 줄: 처음부터 RDF/OWL 중심으로 만들기보다 **"개념 모델은 온톨로지답게 엄격하게, 실제 저장·조회는 속성 그래프(property graph)와 Cypher로 단순하게"** 시작하는 편이 줌줌투어의 시스템 영향도·업무 프로세스·테이블 의존성 질의에 더 적합합니다. RDF는 주어-술어-목적어의 표준 그래프이고, OWL은 그 위에 클래스·속성과 논리적 의미를 정의하는 언어입니다. [2]
핵심 결론 세 줄: DB 스키마 자동 수집과 조직용 카탈로그 UI가 중요해지면 **DataHub 또는 OpenMetadata**가 강력한 대안입니다. 다만 OpenMetadata의 현재 공식 운영 권고는 서버·DB·검색엔진만 합쳐도 상당한 자원이 필요하며, OpenMetadata 본체와 달리 현재 `openmetadata-mcp` 모듈은 별도의 Collate Community License를 사용하므로 반드시 구분해야 합니다. [3]
핵심 결론 네 줄: 무료 Neo4j Community의 가장 큰 약점은 세밀한 RBAC·클러스터·온라인 백업입니다. 이를 **사용자에게 DB 쓰기 권한을 전혀 주지 않고 → MCP는 읽기 전용 → 모든 변경은 Git PR → 승인된 CI 서비스 계정만 그래프를 재생성/갱신**하는 방식으로 보완하면 1차 범위에서는 오히려 단순하고 안전합니다. Community Edition은 한 개의 표준 DB만 지원하고 온라인 백업은 Enterprise 기능입니다. [4]
핵심 결론 다섯 줄: 첫 파일럿은 **"예약 취소 → 환불 → 정산 반영"처럼 예약·CS·결제·외부 PG·정산을 한 번에 가로지르는 프로세스 하나**를 추천합니다. 성공 여부는 "이 테이블을 바꾸면 어느 시스템·업무 단계·부서가 영향을 받는가?" 같은 역량 질문 20개에 대해 정확한 근거와 함께 답하는지로 판단해야 합니다.

> **전제:** 질문에 적힌 직원 수, 시스템 수, DB 수, 서버 사양, 인증 수단, 기존 문서, 담당 인력은 모두 예시 값으로 제시되어 있으므로 현재 실제 환경은 **미정**으로 취급했습니다. 아래 서버 사양·기간·확장 기준 중 벤더 공식 수치가 아닌 값은 명시적으로 **본 조사 권고값**으로 표시합니다.

개념과 설계 방향

온톨로지·지식그래프·그래프 DB를 쉽게 구분하면

세 가지를 같은 것으로 생각하면 설계가 꼬이기 쉽습니다.

개념	쉬운 비유	줌줌투어에서의 예
온톨로지 (Ontology)	도시의 "설계 규칙 + 용어사전"	`System`, `Table`, `Department`, `ProcessStep`이 무엇인지, `ProcessStep READS_FROM Table` 같은 관계가 무엇을 뜻하는지 정의
지식그래프 (Knowledge Graph)	실제 건물과 도로가 그려진 도시 지도	"예약시스템 → reservation DB → booking 테이블", "취소 검증 단계 → booking 읽음 → CS팀 담당" 같은 실제 회사 지식
그래프 DB	그 지도를 저장하고 빠르게 탐색하는 데이터베이스 엔진	Neo4j, Apache AGE, JanusGraph, Jena/Fuseki 등

개념	쉬운 비유	줌줌투어에서의 예
데이터 카탈로그	회사 데이터 자산의 도서관 목록	DB·스키마·테이블·컬럼·오너·설명·리니지 자동 수집
MCP	AI 에이전트가 그 도서관/지도에 접속하는 표준 플러그	Claude Code·Cursor 등이 그래프를 자연어로 조회

즉, **"Neo4j를 설치했다 = 온톨로지가 생겼다"가 아닙니다.** Neo4j는 저장소이고, 줌줌투어가 어떤 종류의 객체와 관계를 관리할지를 먼저 정의한 것이 온톨로지입니다.

속성 그래프와 RDF/OWL

속성 그래프(Property Graph)는 노드와 관계 모두에 자유롭게 속성을 붙이는 모델입니다. 예를 들어:

```
(:ProcessStep {
  name: "환불 가능 여부 확인",
  sequence: 2
})
  -[:READS_FROM {accessType: "SELECT"}]->
(:Table {
  name: "booking"
})
```

사람이 보기 쉽고, "A를 바꾸면 무엇이 영향을 받는가?"처럼 관계를 따라가는 질의가 직관적입니다. Neo4j에서는 Cypher로 조회합니다.

반면 **RDF(Resource Description Framework)**는 모든 지식을 기본적으로 `주어 → 술어 → 목적어` 형태의 트리플로 표현합니다. W3C RDF 표준은 IRI, 리터럴, blank node 등을 이용해 이런 그래프를 정의하고, SPARQL이 표준 질의 언어입니다. [5]

```
zoom:CancellationStep
    zoom:readsFrom
zoom:BookingTable
```

OWL(Web Ontology Language)은 RDF 위에서 `System은 SoftwareAsset의 하위 클래스`, `모든 ProcessStep에는 수행 Role이 있어야 한다`와 같은 의미 체계와 추론 가능한 모델을 더 정교하게 정의하는 W3C 온톨로지 언어입니다. [6]

둘 중 하나가 "진짜 온톨로지"이고 다른 하나가 가짜인 것은 아닙니다. 중요한 차이는 **표준 의미론·교환성·추론을 얼마나 중시하느냐와 개발자가 얼마나 쉽게 운영할 수 있느냐**입니다.

줌줌투어에는 무엇이 맞는가

1차 구축의 물리 저장 모델은 속성 그래프를 권장합니다.

현재 핵심 질문은 다음처럼 대부분 그래프 탐색 문제입니다.

"`booking.status` 컬럼을 변경하면 어느 시스템과 업무 단계가 영향을 받는가?"

"외부 PG 장애가 나면 어떤 프로세스와 담당 부서가 영향을 받는가?"

"예약 취소 단계에서 읽고 쓰는 테이블과 API는 무엇인가?"

"정산팀이 사용하는 테이블을 생성하거나 갱신하는 시스템은 무엇인가?"

이런 질의는 속성 그래프와 Cypher의 강점에 정확히 들어맞고, 비전문가도 그림으로 이해하기 쉽습니다. 최신 Neo4j는 Cypher를 기본 그래프 질의 언어로 제공하며, 2026.10 기준 공식 최신 버전은 2026.10.1입니다. [7]

그러나 **논리적 설계 자체는 RDF 세계의 장점을 가져오는 것이 좋습니다.** 즉,

```
W3C/OMG 표준 용어
        ↓ 의미 차용·매핑
줌줌투어 온톨로지
        ↓
Neo4j 속성 그래프로 물리 구현
```

으로 설계합니다.

이렇게 하면 지금은 Cypher와 MCP를 간단히 쓰면서, 나중에 외부 표준 교환이나 RDF 추론 요구가 생기면 RDF/OWL로 변환할 길도 열어둘 수 있습니다.

OWL부터 시작하지 않는 이유는 현재 문제가 "자동 추론이 없어서" 생긴 것이 아니라 **회사 곳곳에 흩어진 시스템·DB·업무·담당자·프로세스 관계를 일관된 ID와 근거로 연결하지 못해서** 생긴 문제에 가깝기 때문입니다. 처음부터 정교한 OWL 클래스 계층과 추론 규칙을 설계하면 모델링 학습비용이 실제 데이터 수집보다 커질 가능성이 높습니다.

솔루션 비교와 라이선스

2026년 10월 25일 기준으로 그래프 저장소, RDF, 온톨로지 편집기, 데이터 카탈로그, LLM 지식그래프 도구를 섞어 조사했습니다.

후보와 최신 버전

솔루션	종류	현재 확인한 버전	데이터 모델 / 질의	라이선스·사내 상업 이용
Neo4j Community	속성 그래프 DB	**2026.10.1** [8]	Property Graph / Cypher	**GPLv3**. Neo4j가 명시적으로 회사 내부·방화벽 안에서 폐쇄 소프트웨어가 접근하는 Community 사용을 무료로 허용한다고 설명합니다. [9]
Apache Jena + Fuseki	RDF triple/quad store	**6.2.0**, Java 21+ [10]	RDF / SPARQL 1.1	Apache 프로젝트, Apache-2.0 계열. 사내 상업 이용 가능.

솔루션	종류	현재 확인한 버전	데이터 모델 / 질의	라이선스·사내 상업 이용
Protégé Desktop / WebProtégé	OWL 온톨로지 편집기	Desktop **5.6.9 stable**, 6.0.0-alpha-1은 pre-release [11]	RDF/OWL	Desktop·WebProtégé 모두 BSD 계열의 허용적 라이선스. [12]
OpenMetadata	데이터 카탈로그·거버넌스	**2.0.6**, 2026-10-21 릴리스 [13]	자체 메타데이터 모델/API	**본체 Apache-2.0**. 다만 `openmetadata-mcp`는 별도의 **Collate Community License Agreement 1.0**을 사용. [14]
DataHub	데이터 카탈로그·메타데이터 그래프	**v1.8.0**, 2026-10-22 availability [15]	Metadata graph / GraphQL·API	OSS는 **Apache-2.0**. [16]
Apache AGE	PostgreSQL 속성 그래프 확장	**1.7.0**, 2026-01-21 [17]	PostgreSQL + Cypher 계열	Apache 프로젝트. 사내 상업 이용에 적합한 Apache-2.0 계열.
JanusGraph	분산 속성 그래프 DB	안정판 **1.1.0**; 2026년 기준에도 이것이 안정 릴리스로 확인됨 [18]	Property Graph / Gremlin	Apache-2.0. [19]
Graphiti	LLM용 temporal KG 프레임워크	**0.31.2**, MCP 1.1.2 포함 [20]	Neo4j/FalkorDB 등의 그래프 위에 시간성 KG 구축	Graphiti 자체 **Apache-2.0**. [21]

WebProtégé 버전은 확인 필요: 현재 프로젝트 자체는 활발하고 Docker 배포 자료도 있지만, 기존 단일 저장소와 새 배포 구조가 병행되는 전환 상황이 보여 "현재 안정 self-host 배포판 번호"는 도입 시점에 다시 확인하는 것이 안전합니다. WebProtégé는 브라우저 기반 협업·논의·변경 이력·OWL 편집 기능을 제공하며 Docker 자체 호스팅 자료가 있습니다. [22]

기능·운영 비교

난이도는 **★가 많을수록 어렵다**는 본 조사 정성평가입니다.

항목	Neo4j Community	Jena/Fuseki	Protégé/ WebProtégé	OpenMetadata
주 역할	전사 관계 그래프	표준 RDF 저장소	온톨로지 설계·협업	자동 데이터 카탈로그
비전문가 학습	★★☆	★★★★	★★★★	사용자 ★★ / 운영 ★★★★

항목	Neo4j Community	Jena/Fuseki	Protégé/ WebProtégé	OpenMetadata
웹 UI/ 시각화	Neo4j Browser 포함 [23]	Fuseki UI는 관리·SPARQL 중심	매우 강함, 특히 OWL 편집	매우 강한 카탈로그 UI
다중 사용자	가능하나 CE 권한 모델 약함	서버·dataset·endpoint ACL 제공 [24]	공동 편집·토론·변경이력 강점 [25]	팀/역할/정책 중심 거버넌스 강함
세밀한 RBAC	**Community 약점**. 고급 RBAC는 Enterprise 기능군 [26]	ACL 가능. graph-level ACL은 현재 read-only dataset에 적용 [24]	프로젝트 협업 권한 중심	강함
MCP	**공식**	공식 MCP 미확인	공식 MCP 미확인	공식 MCP 존재
MCP 읽기 전용	**명확한 전역 스위치 있음** `NEO4J_READ_ONLY=true` [27]	커스텀 MCP 필요	커스텀 필요	전체 MCP 서버를 한 번에 read-only로 만드는 공식 스위치는 이번 조사에서 **확인 필요**
DB 스키마 자동 수집	기본 없음	없음	없음	**매우 강함**
Docker	지원	공식 Docker 자료 있음 [28]	WebProtégé Docker	지원
백업	CE는 **offline dump**, 온라인 백업은 Enterprise [29]	TDB2는 live backup 가능 [30]	ontology export + 데이터 백업	underlying DB/ search 백업 필요
클러스터/HA	Community는 사실상 단일 인스턴스, 클러스터는 Enterprise 영역	별도 아키텍처 필요	목적 외	가능하지만 복잡
커뮤니티	매우 큼	오래되고 안정적	학술·정부·의료 분야 매우 성숙	큼
한국어	비공식 자료 일부, 최신 공식 한글 문서는 본 조사에서 미확인	매우 적음	매우 적음	적음

OpenMetadata의 MySQL 커넥터는 메타데이터 수집 시 schema/table discovery, column 이름·자료형·comment, PK/FK, index, DDL 등을 수집할 수 있고, 문서에는 **SELECT 없이도 메타데이터 수집**하는 설정이 별도로 설명되어 있습니다. 이는 “실제 고객 데이터를 절대 읽지 않는다”는 줌줌투어 요구와 특히 잘 맞습니다. PostgreSQL 커넥터도 schema/table/column 등 자동 메타데이터 수집을 지원합니다. [31]

항목	DataHub	Apache AGE	JanusGraph	Graphiti
주 역할	데이터 카탈로그·리니지	PostgreSQL 안의 그래프	대규모 분산 그래프	AI가 지식을 추출·갱신하는 KG
학습	★★★	★★★	★★★★★	★★★★
UI	강한 카탈로그 UI	PostgreSQL 도구 위주	자체 비즈니스 UI 약함	최종 사용자용 카탈로그 UI가 목적이 아님
권한	역할·정책 지원	PostgreSQL 권한 활용, 1.7.0에 RLS 지원 추가 [17]	backend/app 설계 의존	**reference 서버는 엔터프라이즈 권한계층 용도가 아님**
MCP	**공식 self-host MCP** [32]	공식 미확인	공식 미확인	공식 프로젝트에 **experimental reference MCP** 포함 [33]
Read/Write 구분	도구가 read-only/write로 분류되어 MCP hints 제공 [32]	별도 구현	별도 구현	삭제·추가 도구가 함께 노출; 전사 read-only 용도로 부적합
자동 스키마 수집	**매우 강함**, DB connector 다수	직접 구현	직접 구현	없음
Docker	Quickstart 지원, 프로덕션에는 그대로 쓰지 말라고 공식 경고 [34]	공식 Docker 실행법 제공 [35]	공식 container·Compose 자료 제공 [36]	Docker 지원
운영 복잡도	높음	낮음~중간	매우 높음	중간~높음
백업	메타데이터 DB + 검색 인덱스 운영 필요	PostgreSQL 표준 백업 사용	저장 backend별로 상이	Neo4j/FalkorDB backend 백업에 의존
커뮤니티	매우 큼	중간	중간	AI/KG 분야 매우 활발; 현재 GitHub 저장소 규모도 큼 [37]
한국어	공식 최신 한국어 문서 본 조사에서 미확인	적음	적음	적음

라이선스에서 특히 조심할 점

Neo4j Community의 GPLv3는 줌줌투어의 "사내 서버 내부 이용" 자체를 막지 않습니다. Neo4j 공식 라이선스 설명은 Community Edition을 방화벽 안에서 회사 애플리케이션이 사용하는 경우, 그 애플리케이션이 폐쇄소스여도 무료 사용이 가능하다고 명시합니다. 배포·수정본 재배포 시에는 GPL 의무를 별도로 검토해야 하지만 이번 "사내용 지식 그래프 DB" 시나리오는 공식 설명과 잘 맞습니다. [9]

OpenMetadata는 라이선스를 한 줄로 "Apache 2.0"이라고 적으면 현재는 부정확합니다. 메인 저장소는 Apache-2.0이지만 `openmetadata-mcp` 하위 모듈에는 Collate Community License Agreement 1.0이 들어 있습니다. 그 라이선스는 사용·수정 등의 라이선스를 부여하면서도 Collate 제품과 경쟁하는 SaaS/PaaS/IaaS 형태의 서비스 제공을 `Excluded Purpose`로 금지합니다. 사내 내부 이용은 문언상 경쟁 서비스 제공과는 다르므로 일반적으로 충돌하지 않아 보이지만, **이는 법률 자문이 아니며 MCP까지 채택한다면 사내 법무 확인을 권고합니다.** [38]

Graphiti 자체는 Apache-2.0이지만 "무료니까 그대로 전사 MCP로 공개"해서는 안 됩니다. 프로젝트 README는 reference MCP/REST 서버가 production용 인증·인가·tenant isolation 등을 대신해 주지 않는다는 취지로 명시하며, production에서는 `graphiti-core`를 자체 서비스에 넣어 이러한 통제를 직접 구현하라고 안내합니다. 최신 reference MCP는 실험적이며 추가·삭제·clear graph 같은 쓰기 도구도 제공합니다. [39]

서버 자원 관점

가장 큰 차이는 OpenMetadata/DataHub 같은 **카탈로그 플랫폼 전체**와 Neo4j 같은 **단일 그래프 DB**를 비교할 때 나타납니다.

OpenMetadata 2.0 공식 production 권고는 OpenMetadata 서버 자체에 4 vCPU/16 GiB, DB에 4 vCPU/16 GiB, Elasticsearch/OpenSearch에 2 vCPU/8 GiB를 제시하며, 별도의 Airflow ingestion 환경에도 4 vCPU/16 GiB 권고가 있습니다. 즉 모든 것을 같은 한 대에 얹는다면 상당한 자원이 필요합니다. [40]

반면 줌줌투어 1차 지식그래프가 **실데이터가 아닌 시스템·테이블·컬럼·업무 메타데이터**만 담는다면 데이터량 자체는 매우 작을 가능성이 큽니다. 따라서 다음은 벤더 최소사양이 아니라 **본 조사에서 제안하는 파일럿 시작값**입니다.

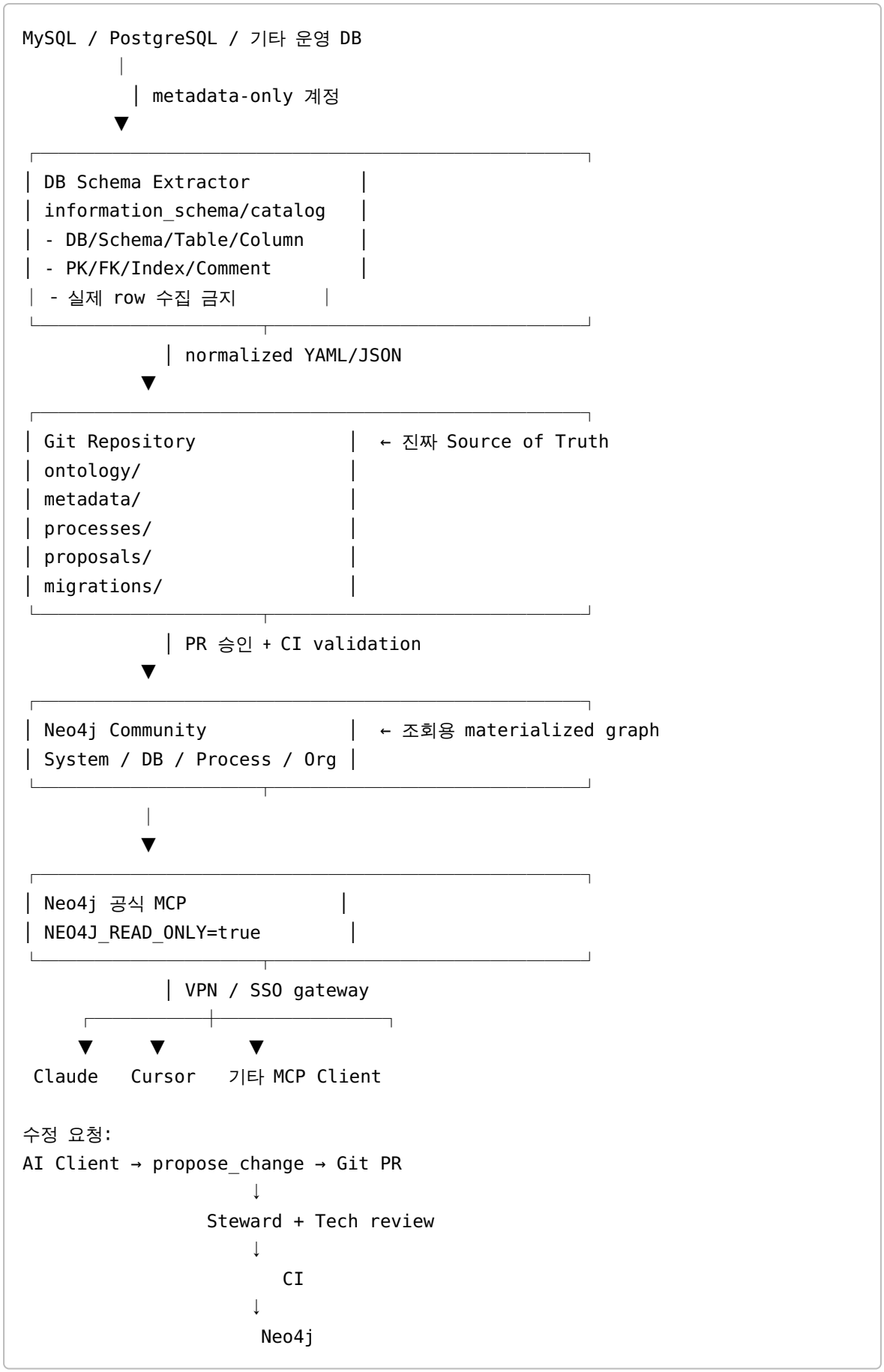
구성	파일럿 권고 시작값	비고
Neo4j CE + MCP + extractor	4 vCPU / 8–16 GB RAM / SSD 50 GB	실제 엔티티 수에 따라 조정
Jena + Fuseki	4 vCPU / 8–16 GB	RDF reasoning을 많이 쓰면 추가
AGE + PostgreSQL	4 vCPU / 8–16 GB	기존 PostgreSQL 운영 경험이 있으면 유리
DataHub	8 vCPU / 16–32 GB 이상을 파일럿 시작점으로 검토	Quickstart는 production용이 아님 [34]
OpenMetadata	**공식 production 권고에 맞춰 별도 용량 산정 필요**	작은 사내 VM 한 대에는 부담될 수 있음 [41]

동시 쿼리 수: 조사한 OSS 제품 대부분은 "무료판은 동시 쿼리 N개" 같은 라이선스 캡을 공식적으로 두기보다 CPU·메모리·connection pool·backend 성능에 제한됩니다. 따라서 특정 숫자를 무료판 제한으로 적는 것은 부정확합니다. Neo4j Community 역시 이번 공식 문서 조사에서 무료판 고정 동시쿼리 수 제한은 확인하지 못했습니다. **확인 필요 항목으로 유지하는 것이 맞습니다.**

권장 아키텍처와 보안

일순위 구성

가장 추천하는 형태는 다음입니다.


MySQL / PostgreSQL / 기타 운영 DB
metadata-only 계정
DB Schema Extractor
information_schema/catalog
- DB/Schema/Table/Column
- PK/FK/Index/Comment
- 실제 row 수집 금지
normalized YAML/JSON
Git Repository
ontology/
metadata/
processes/
proposals/
migrations/
← 진짜 Source of Truth
PR 승인 + CI validation
Neo4j Community
System / DB / Process / Org
← 조회용 materialized graph
Neo4j 공식 MCP
NEO4J_READ_ONLY=true
VPN / SSO gateway
Claude
Cursor
기타 MCP Client
수정 요청:
AI Client → propose_change → Git PR
Steward + Tech review
CI
Neo4j

Neo4j의 현재 공식 MCP는 Claude Code와 Claude Desktop 설정 예시를 직접 제공하며, `NEO4J_READ_ONLY=true`로 쓰기 도구를 노출하지 않을 수 있습니다. `read-cypher`는 쓰기·관리·schema operation을 거부하고 `write-cypher`는 read-only 모드에서 아예 노출되지 않습니다. [42]

이 구조에서 **"그래프 DB는 원본이 아니라 Git으로부터 재생성할 수 있는 결과물"**로 취급하는 것이 핵심입니다.

그렇게 하면 Neo4j Community의 온라인 백업 부재도 상당 부분 상쇄됩니다. Community는 offline dump만 제공하지만, 모든 모델과 수집된 스키마 snapshot이 Git/백업 스토리지에 있다면 장애 시 새 Neo4j를 올려 재적재할 수 있습니다. 그래프 DB 자체의 offline dump도 주기적으로 보관하면 됩니다. [43]

왜 다른 후보를 일순위에서 제외했는가

OpenMetadata는 매우 좋은 제품이지만 1차에서는 다소 큽니다. 스키마 자동 수집, 검색, owner, glossary, lineage, RBAC가 잘 갖춰져 있어 "데이터 카탈로그"가 핵심 목표라면 Neo4j보다 오히려 좋습니다. 그러나 줌줌투어의 범위는 DB 카탈로그를 넘어 `부서 → 업무 → ProcessStep → System → Table → 외부 API`를 자유롭게 연결하는 것이 핵심이고, 공식 production 자원 요구도 큽니다. [44]

DataHub 역시 데이터 자산 중심으로는 훌륭하고 공식 self-host MCP까지 있습니다. MCP에서는 lineage, schema, 검색을 자연어 에이전트에 제공하고 도구를 read/write로 구분합니다. 하지만 초기 목표가 "회사 전체 업무·시스템 온톨로지를 자유롭게 확장"하는 것이라면 카탈로그의 기존 메타모델에 맞추는 부담이 생길 수 있습니다. [32]

Jena/Fuseki + Protégé는 의미론적 정합성과 표준성이 가장 좋습니다. Fuseki는 SPARQL 서버이고 ACL과 live TDB2 backup도 지원합니다. Protégé는 OWL 2 전체 생명주기를 위한 대표적 오픈소스 편집기입니다. 하지만 첫 구축 담당자가 온톨로지 비전문가이고 AI 에이전트 질의가 핵심인 현재 조건에서는 RDF/OWL/SPARQL의 학습비용을 감수할 직접적인 이득이 아직 작습니다. [45]

Apache AGE는 매력적인 경량 대안입니다. PostgreSQL 확장이므로 기존 PostgreSQL 운영 능력을 그대로 활용할 수 있고 SQL과 그래프를 한 서버에 둘 수 있습니다. 1.7.0에는 RLS도 추가됐습니다. 다만 Neo4j에 비해 그래프 생태계·시각화·공식 MCP 연결의 완성도가 낮아, "AI 에이전트가 쓰는 사내 그래프"라는 요구에서는 Neo4j 쪽이 편합니다. [46]

JanusGraph는 지금 규모에는 과합니다. Cassandra/Elasticsearch 같은 backend와 함께 매우 큰 분산 그래프를 운영하는 데 유리하지만, 회사 시스템·스키마 메타데이터 규모에서는 운영 복잡성 대비 이익이 없습니다. 공식 Docker 구성에서도 Cassandra와 Elasticsearch를 조합한 배포 패턴이 나타납니다. [36]

Graphiti는 1차의 주 저장소로 쓰지 않는 것이 좋습니다. LLM으로 시간에 따라 바뀌는 지식을 추출·갱신하는 프레임워크라는 점은 흥미롭지만, reference MCP가 실험적이고 production 권한 제어를 애플리케이션이 직접 책임져야 합니다. 향후 "문서·대화에서 후보 지식을 자동 추출해 변경 제안으로 만드는 보조 에이전트"로 쓰는 편이 더 적합합니다. [39]

현실적인 대안 구성

순위	구성	적합한 경우
일순위	**Neo4j CE + Git + custom metadata extractor + 공식 Neo4j MCP + 선택적 Protégé Desktop**	작은 팀, 빠른 시작, 업무/시스템 영향 분석 중심

순위	구성	적합한 경우
대안 A	**DataHub + 공식 DataHub MCP + Git으로 별도 업무 ontology 관리**	DB·대시보드·lineage·catalog UX가 업무 ontology보다 중요해질 때
대안 B	**Apache Jena/Fuseki + Protégé/WebProtégé + custom extractor + custom MCP**	외부기관 교환, RDF/OWL 표준, formal reasoning이 핵심 요구가 될 때
조건부	OpenMetadata 2.0.6	자동 ingestion·governance UI·RBAC를 한 제품에 많이 맡기고 운영자원도 충분할 때

Community Edition의 권한 약점을 운영으로 보완하는 법

Neo4j CE에 팀원별 세밀한 그래프 권한을 억지로 구현하지 않습니다.

사람에게 Neo4j DB 계정을 주지 않는 것이 가장 단순합니다.

일반 팀원은:

```
사용자
  ↓
회사 VPN / 인증 Gateway
  ↓
읽기 전용 MCP
  ↓
Neo4j read account
```

만 허용합니다.

변경은:

```
"이 설명을 수정해 줘"
       ↓
AI agent가 DB를 UPDATE하지 않음
       ↓
proposals/2026-10-xxx.yaml 생성
       ↓
Git PR
       ↓
Steward 검토
       ↓
Merge
       ↓
CI의 전용 writer 계정
```

```
        ↓
Neo4j 반영
```

으로 고정합니다.

이렇게 하면 **무료판 DB 자체에 강력한 RBAC가 없어도 "사람은 read-only, CI만 write"라는 매우 작은 권한 표면**을 만들 수 있습니다.

원천 DB 보안

원천 DB마다 전용 계정을 둡니다.

```
kg_catalog_reader_reservation
kg_catalog_reader_payment
kg_catalog_reader_settlement
```

권장 원칙은 다음과 같습니다.

항목	권장 정책
접속 출발지	schema-extractor 서버 IP만 허용
쓰기 권한	INSERT/UPDATE/DELETE/DDL/EXECUTE 금지
실제 row SELECT	원칙적으로 금지
수집	DB/schema/table/column/PK/FK/index/comment/type
sample data	수집 금지
profiler	1차 범위에서 비활성
query history	PII가 SQL literal에 포함될 가능성이 있어 1차에서는 비활성
credential	Git 금지, Docker secret/권한 제한 파일 등 사용
운영/개발 DB	반드시 계정·secret 분리

특히 OpenMetadata MySQL 문서는 SELECT 없이도 schema/table/column/key/index/DDL과 같은 메타데이터를 수집하는 방식을 공식적으로 설명하므로, 향후 OpenMetadata를 도입한다면 그 모드를 우선 검토할 가치가 있습니다. [47]

MCP 보안

HTTP 기반 MCP는 protocol 수준에서 OAuth 기반 authorization 구조를 정의하고 있으며, 최신 MCP 보안 가이드는 prompt injection, session hijacking, 과도한 scope 등의 공격을 별도로 다룹니다. [48]

줌줌투어에서는 다음 경계를 권장합니다.

```
Internet
   X
```

```
   |
Company VPN / Zero-Trust network
   |
SSO/OIDC reverse proxy
   |
MCP Gateway
   ├── /kg-read        → Neo4j MCP READ ONLY
   └── /kg-propose     → Git proposal service
```

Neo4j Bolt 포트는 직원 PC에 직접 공개하지 않습니다.

MCP audit log에는 최소한:

```
timestamp
userId / SSO subject
clientId
MCP tool
query hash
target graph
result count
latency
decision: allow/deny
```

를 기록합니다. 결과 데이터 전체를 로그에 남기는 것은 오히려 정보 노출을 늘릴 수 있으므로 기본값으로 권하지 않습니다.

외부 클라우드 LLM 사용 위험

개인정보를 넣지 않는다고 해서 그래프가 "공개 가능한 데이터"가 되는 것은 아닙니다.

다음 정보도 충분히 민감한 회사 구조정보입니다.

```
운영 시스템 이름
내부 DB/테이블/컬럼 이름
외부 PG·공급사 구조
API endpoint
인증 흐름
장애 영향 관계
어느 부서가 어느 시스템을 통제하는지
정산/환불 로직
```

AI 클라이언트가 클라우드 LLM을 사용할 경우, MCP 결과의 일부가 모델 제공자에게 전송될 수 있으므로 **계약·DPA·보존기간·학습 사용 여부·region을 해당 플랜별로 직접 확인해야 합니다.**

기술적으로는 다음 방식을 권장합니다.

```
그래프
 ↓
MCP가 필요한 subgraph만 반환
 ↓
secret / hostname / credential 필터
 ↓
LLM
```

“그래프 전체를 한 번에 context로 제공”해서는 안 됩니다.

민감도가 높은 조회에는 **로컬 LLM**을 별도 선택지로 둘 수 있습니다. Ollama 자체 런타임은 MIT License로 공개되어 있어 사내 inference 계층으로 사용하기 쉽습니다. 단, Ollama 위에서 실행하는 **모델 자체의 라이선스는 각각 별도**이므로 반드시 모델별 확인이 필요합니다. [49]

1차에서는 vector embedding이나 GraphRAG를 반드시 넣을 필요도 없습니다. 대부분의 핵심 질문은 **정형 Cypher traversal**로 정확하게 해결할 수 있어, 민감한 메타데이터를 임베딩 API로 보내는 불필요한 경로를 만들지 않는 편이 낫습니다.

온톨로지 설계안과 예시 코드

가장 중요한 모델링 원칙

실제 고객, 실제 예약, 실제 결제 한 건을 그래프 노드로 만들지 않습니다.

예를 들어:

```
(:BusinessEntity {name:"Booking"})
```

은 “예약이라는 업무 개념”이지,

```
(:Booking {bookingId:"ZZ1234", customerName:"홍길동"})
```

같은 실제 예약 레코드가 아닙니다.

그래프에 들어갈 것은 **메타데이터와 업무 정의**입니다.

권장 클래스

영역	클래스	의미
시스템	`System`	예약, 파트너어드민, 정산 등 독립 운영 시스템
	`Application`	배포 가능한 애플리케이션
	`Module`	애플리케이션 내부 모듈
	`APIEndpoint`	내부/외부 API endpoint 정의

영역	클래스	의미
	`Integration`	두 시스템 사이 연동
	`ExternalService`	PG, 공급사 API, SaaS
	`BatchJob`	정기/비동기 작업
	`MessageTopic`	Kafka/SQS 등의 이벤트 channel
데이터	`Database`	물리/논리 DB
	`DbSchema`	schema/catalog
	`Table`	테이블/view 메타데이터
	`Column`	컬럼
	`PrimaryKey`	PK 정의
	`ForeignKey`	FK 정의
	`Index`	인덱스
	`BusinessEntity`	Booking, Payment 등 업무 개념
조직	`Department`	CS, 정산, 개발 등
	`Team`	하위 조직
	`Role`	CSAgent, SettlementOperator 같은 역할
업무	`BusinessProcess`	예약 생성, 취소·환불 등
	`ProcessStep`	프로세스 한 단계
	`Decision`	분기
	`Trigger`	프로세스를 시작시키는 사건
	`BusinessRule`	환불 가능 조건 등
여행	`TourProduct`	투어/액티비티 상품이라는 개념
	`ProductOption`	날짜·옵션 등의 개념
	`Availability`	재고/가능 여부 개념
	`Booking` 또는 `BookingConcept`	예약 도메인 개념
	`Payment`	결제 개념
	`Refund`	환불 개념
	`Customer`	고객 개념만, 고객 instance 금지
	`Partner`	판매/업무 파트너 개념·회사
	`Supplier`	현지 투어 공급사

영역	클래스	의미
	`Settlement`	정산
	`CSCase`	고객 문의/사건 개념
	`Voucher`	바우처
거버넌스	`SourceArtifact`	ERD, DDL, 문서, 인터뷰 등 근거
	`Verification`	검증 기록
	`ChangeProposal`	변경 제안

`Booking`, `Payment` 같은 여행 도메인은 실제 transaction node가 아니라 **business concept/type**로만 관리하는 것이 안전합니다. 구현에서는 혼동 방지를 위해 `BusinessEntity{name:"Booking"}` 형태가 더 좋습니다.

핵심 관계

관계	예
`CONTAINS`	Database → Table, Table → Column
`HAS_COLUMN`	Table → Column
`FK_TO`	Column → Column
`USES_DATABASE`	System → Database
`OWNS`	Department → System
`STEWARDS`	Role/Department → Entity
`HAS_MODULE`	System → Module
`CALLS_API`	System/Module → APIEndpoint
`HAS_INTEGRATION`	System → Integration
`TARGETS`	Integration → ExternalService
`DEPENDS_ON`	System → System
`REPRESENTS`	Table → BusinessEntity
`HAS_STEP`	BusinessProcess → ProcessStep
`PRECEDES`	ProcessStep → ProcessStep
`PERFORMED_BY`	ProcessStep → Role
`USES_SYSTEM`	ProcessStep → System
`READS_FROM`	ProcessStep → Table/Column
`WRITES_TO`	ProcessStep → Table/Column

관계	예
`TRIGGERS`	Trigger → Process
`PRODUCES`	Step → BusinessEntity
`CONSUMES`	Step → BusinessEntity
`IMPLEMENTS_RULE`	Step/System → BusinessRule
`DERIVED_FROM`	Entity → SourceArtifact
`VERIFIED_BY`	Entity → Verification/Role
`AFFECTS`	Change/Integration → System/Process

관계를 너무 세분화하는 것도 피합니다. 예를 들어 첫 버전부터 `SELECTS_FROM`, `INSERTS_INTO`, `UPDATES`, `DELETES_FROM` 네 관계로 나누기보다:

```
(step)-[:READS_FROM]->(table)

(step)-[:WRITES_TO {operation:"UPDATE"}]->(table)
```

가 관리하기 쉽습니다.

이름 규칙

항목	규칙	예
클래스/Label	영어 PascalCase	`ProcessStep`
관계	영어 UPPER_SNAKE_CASE	`READS_FROM`
property	camelCase	`lastVerifiedAt`
canonical ID	절대 변경하지 않는 소문자 URI-like ID	`sys:reservation`
DB ID	환경+DB+schema+object가 구분되도록	`table:reservation-prod.public.booking`
사람이 보는 이름	`nameKo`, `nameEn` 허용	`nameKo:"예약"`
약어	glossary에 먼저 정의	PG, CMS 등
삭제된 자산	ID 재사용 금지	`status:"retired"`

사람이 바꾸는 이름과 ID를 분리해야 합니다.

예:

```
uid: "sys:reservation"
nameKo: "예약 시스템"
displayName: "Booking"
```

나중에 “예약 시스템”이 “Core Booking”으로 이름을 바꿔도 `uid`는 그대로입니다.

모든 관리 객체의 필수 속성

최소 권장값은 다음입니다.

```
uid: "table:reservation.public.booking"
name: "booking"
description: "예약의 현재 상태와 핵심 식별자를 관리하는 테이블"
status: "active"

ownerTeam: "team:booking"
steward: "role:booking-data-steward"

sourceType: "db-introspection"
sourceRef: "source:reservation-prod-2026-10-25"

verifiedBy: "role:booking-tech-owner"
lastVerifiedAt: "2026-10-25"
updatedAt: "2026-10-25"

confidence: "verified"
sensitivity: "internal"
```

`updatedAt`과 `lastVerifiedAt`은 다릅니다.

updatedAt은 마지막으로 파일/그래프가 바뀐 시각이고,
lastVerifiedAt은 사람이 “지금도 사실”이라고 확인한 시각입니다.

이 차이가 없으면 오래된 정보가 수정되지 않았다는 이유로 “최신”처럼 보이는 문제가 생깁니다.

표준 어휘 재사용

새로 모든 개념을 발명하지 말고 기존 표준과 의미를 매핑합니다.

표준	가져올 개념	활용
W3C ORG	Organization, OrganizationalUnit, Role, Membership	Department·Team·Role 의미 정의. 조직 온톨로지를 처음부터 자체 발명하지 않음. [50]

표준	가져올 개념	활용
PROV-O	Entity, Activity, Agent, wasDerivedFrom, wasGeneratedBy	“이 지식이 어느 DB/문서/인터뷰에서 왔는가” provenance 기록. [51]
DCAT 3	Dataset, DataService, Catalog	DB 데이터셋·API·카탈로그 개념 매핑. [52]
SKOS	Concept, ConceptScheme, broader/narrower, prefLabel	업무 용어집, 카테고리, 동의어 관리. [53]
DCMI Terms	title, description, source, modified, identifier	공통 문서/메타데이터 속성. DCMI는 RDF 외 환경에서도 용어의 자연어 의미를 재사용할 수 있다고 설명합니다. [54]
schema.org	Organization, SoftwareApplication 등	외부 서비스·회사·software 개념과 교환성 확보. [55]
BPMN 2.0.2	Process, Task, Event, Gateway, Pool/Lane	프로세스 인터뷰·다이어그램 표현 기준. BPMN은 OMG 표준입니다. [56]
SHACL	Shape/constraint	나중에 RDF로 갈 때 필수 property/cardinality 검증. 현재 안정 Recommendation은 SHACL이고, 2026년 10월에는 SHACL 1.2 초안도 진행 중입니다. [57]

특히 **“BPMN 온톨로지”는 주의**해야 합니다. BPMN 자체는 OMG의 업무 프로세스 표준 notation/metamodel이지 W3C 공식 OWL 온톨로지가 아닙니다. 따라서 임의의 커뮤니티 BPMN OWL 파일을 핵심 의존성으로 가져오기보다는 BPMN의 `Process / Task / Gateway / Event / Lane` 개념을 줌줌투어 모델에 매핑하는 방식을 추천합니다. [56]

Neo4j 데이터 입력 예시

아래 데이터는 **예시이며 실제 줌줌투어 시스템 이름·스키마를 나타내지 않습니다.**

먼저 관리 객체의 고유 ID를 보장합니다.

```
CREATE CONSTRAINT managed_entity_uid IF NOT EXISTS
FOR (n:ManagedEntity)
REQUIRE n.uid IS UNIQUE;
```

예시 A — 시스템·DB·테이블·컬럼

```
MERGE (sys:ManagedEntity:System {uid: 'sys:reservation'})
SET sys.nameKo = '예약 시스템',
  sys.status = 'active',
  sys.steward = 'role:booking-tech-owner',
  sys.sourceRef = 'source:system-inventory',
  sys.lastVerifiedAt = date('2026-10-25'),
  sys.updatedAt = date('2026-10-25');
```

```
MERGE (db:ManagedEntity:Database {uid: 'db:reservation-main'})
SET db.name = 'reservation',
  db.dbms = 'mysql',
  db.status = 'active',
  db.sourceRef = 'source:db-introspection';

MERGE (booking:ManagedEntity:Table {
 uid: 'table:reservation.booking'
})
SET booking.name = 'booking',
  booking.description = '예약 업무 엔티티를 저장하는 테이블',
  booking.sourceType = 'db-introspection',
  booking.sourceRef = 'source:reservation-schema',
  booking.lastVerifiedAt = date('2026-10-25');

MERGE (status:ManagedEntity:Column {
 uid: 'column:reservation.booking.status'
})
SET status.name = 'status',
  status.dataType = 'varchar',
  status.description = '예약 상태 코드';

MERGE (sys)-[:USES_DATABASE]->(db);
MERGE (db)-[:CONTAINS]->(booking);
MERGE (booking)-[:HAS_COLUMN]->(status);
```

예시 B — 업무 프로세스와 읽기/쓰기

```
MERGE (process:ManagedEntity:BusinessProcess {
 uid: 'process:booking-cancellation'
})
SET process.nameKo = '예약 취소 및 환불',
  process.status = 'active',
  process.steward = 'role:cs-process-steward';

MERGE (step:ManagedEntity:ProcessStep {
 uid: 'step:booking-cancellation:verify'
})
SET step.nameKo = '취소 가능 여부 확인',
  step.sequence = 1,
  step.sourceType = 'process-workshop',
  step.lastVerifiedAt = date('2026-10-25');

MERGE (role:ManagedEntity:Role {uid: 'role:cs-agent'})
SET role.nameKo = 'CS 담당자';
```

```
MERGE (dept:ManagedEntity:Department {uid: 'dept:cs'})
SET dept.nameKo = 'CS';

MATCH (booking:Table {uid: 'table:reservation.booking'})
MATCH (sys:System {uid: 'sys:reservation'})

MERGE (process)-[:HAS_STEP]->(step);
MERGE (step)-[:PERFORMED_BY]->(role);
MERGE (dept)-[:HAS_ROLE]->(role);
MERGE (step)-[:USES_SYSTEM]->(sys);
MERGE (step)-[:READS_FROM {reason:'취소 가능 상태 확인'}]->(booking);
```

예시 C — 외부 PG와 결제 연동

```
MERGE (payment:ManagedEntity:System {uid: 'sys:payment'})
SET payment.nameKo = '결제 시스템';

MERGE (pg:ManagedEntity:ExternalService {uid: 'external:pg-provider'})
SET pg.nameKo = '외부 PG',
  pg.category = 'payment-gateway',
  pg.sensitivity = 'internal';

MERGE (integration:ManagedEntity:Integration {
 uid: 'integration:payment-authorize'
})
SET integration.nameKo = '결제 승인 연동',
  integration.protocol = 'HTTPS',
  integration.direction = 'outbound',
  integration.status = 'active';

MERGE (concept:ManagedEntity:BusinessEntity {
 uid: 'concept:payment'
})
SET concept.nameKo = '결제';

MERGE (payment)-[:HAS_INTEGRATION]->(integration);
MERGE (integration)-[:TARGETS]->(pg);
MERGE (payment)-[:MANAGES]->(concept);
```

예시 D — FK 메타데이터

```
MATCH
 (supplierId:Column {
  uid:'column:reservation.booking.supplier_id'
 }),
 (partnerId:Column {
```

```
  uid:'column:partner.partner.id'
 })

MERGE (supplierId)-[fk:FK_TO]->(partnerId)
SET fk.constraintName = 'fk_booking_supplier',
  fk.sourceType = 'db-introspection',
  fk.lastVerifiedAt = date('2026-10-25');
```

Cypher 질의 예시

질의 A — 특정 테이블을 바꾸면 영향을 받는 업무·부서는?

```
MATCH (t:Table {uid: $tableUid})

OPTIONAL MATCH
 (process:BusinessProcess)-[:HAS_STEP]->(step:ProcessStep)
 -[rw:READS_FROM|WRITES_TO]->(t)

OPTIONAL MATCH
 (step)-[:PERFORMED_BY]->(role:Role)
 <-[:HAS_ROLE]-(dept:Department)

RETURN
 t.name AS table,
 process.nameKo AS process,
 step.nameKo AS step,
 type(rw) AS access,
 role.nameKo AS role,
 dept.nameKo AS department
ORDER BY process, step;
```

사용자의 예시 질문인 **"예약 테이블을 바꾸면 영향받는 부서는?"**에 직접 대응합니다.

질의 B — 한 프로세스의 단계·담당자·읽기/쓰기 테이블

```
MATCH (p:BusinessProcess {uid: $processUid})
MATCH (p)-[:HAS_STEP]->(step:ProcessStep)

OPTIONAL MATCH (step)-[:PERFORMED_BY]->(role:Role)
OPTIONAL MATCH (step)-[rw:READS_FROM|WRITES_TO]->(t:Table)

RETURN
 step.sequence AS seq,
 step.nameKo AS step,
 role.nameKo AS role,
 type(rw) AS accessType,
```

```
  t.uid AS table
ORDER BY seq, accessType, table;
```

질의 C — 외부 서비스 장애 시 영향 시스템·프로세스

```
MATCH (external:ExternalService {uid: $externalUid})
MATCH (integration:Integration)-[:TARGETS]->(external)
MATCH (system:System)-[:HAS_INTEGRATION]->(integration)

OPTIONAL MATCH
  (process:BusinessProcess)-[:HAS_STEP]->(step:ProcessStep)
  -[:USES_SYSTEM]->(system)

RETURN DISTINCT
  external.nameKo AS externalService,
  integration.nameKo AS integration,
  system.nameKo AS impactedSystem,
  process.nameKo AS impactedProcess;
```

질의 D — 검증이 오래됐거나 담당자가 없는 지식 찾기

```
MATCH (n:ManagedEntity)
WHERE
  n.steward IS NULL
  OR n.sourceRef IS NULL
  OR n.lastVerifiedAt IS NULL
  OR n.lastVerifiedAt < date() - duration('P90D')

RETURN
  n.uid,
  labels(n),
  n.steward,
  n.sourceRef,
  n.lastVerifiedAt
ORDER BY n.lastVerifiedAt ASC;
```

이 마지막 질의가 실제 장기 운영에서는 매우 중요합니다. 지식그래프는 “많이 넣는 것”보다 **무엇이 낡았는지 즉시 찾을 수 있어야** 가치가 유지됩니다.

파일럿 구축 로드맵과 협업 거버넌스

기간은 실제 담당 인력이 미정이므로 **기술 담당자 한 명이 주 약 2일, 각 관련 부서 담당자가 주 1~2시간 협조**한다는 가정의 권고 일정입니다. 이 경우 전체 파일럿은 대략 **8~12주**를 계획하는 것이 현실적입니다. 전담 개발자가 있으면 더 짧아질 수 있습니다.

파일럿 대상

가장 좋은 첫 프로세스 후보는:

예약 취소 → 환불 → 공급사/PG 처리 → 정산 반영

입니다.

이 프로세스가 적합한 이유는 하나의 파일럿에서 다음을 모두 시험할 수 있기 때문입니다.

```
고객/예약 개념
→ CS 부서
→ 예약 시스템
→ 예약 DB
→ 외부 PG
→ 공급사
→ 환불
→ 정산 시스템
→ 정산 부서
```

단순한 “상품 등록”보다 그래프를 왜 만드는지 빨리 증명할 수 있습니다.

단계별 계획

단계	권고 기간	주 담당	산출물	AI 에이전트 활용	완료 기준
범위 정의	3~5일	프로젝트 오너 + 개발	파일럿 scope, 시스템 목록, 역량 질문	기존 문서에서 후보 시스템·질문 추출	포함/제외 범위 승인, 핵심 질문 10~20개 확정
개념 도출	3~5일	데이터 아키텍트 역할 + 현업	glossary, 클래스 후보	문서/ERD에서 명사·업무 용어 후보 추출	중복 용어 정리, 핵심 클래스 합의
관계 정의	3~5일	개발 + 현업 steward	relationship dictionary	후보 관계·방향·cardinality 초안	모든 핵심 관계에 정의·예시 존재
인프라 설치	1~3일	개발/인프라	Docker Compose, Neo4j, MCP, Git repo	Compose·config·CI 작성 지원	내부망 접속, read-only MCP 성공
DB 스키마 자동 추출	1~2주	개발	extractor, schema snapshot	AI가 DB별 introspection 코드·mapping 생성	대상 DB의 table/column/PK/FK 100% 또는 승인된 제외 목록

단계	권고 기간	주 담당	산출물	AI 에이전트 활용	완료 기준
부서·업무·프로세스 모델링	1~2주	현업 steward + 개발	BPMN, step/role/read/write 표	인터뷰 메모 → BPMN/graph 후보 자동 변환	파일럿 모든 step에 담당 role·system·R/W table 연결
적재	3~5일	개발	Cypher/migration, validation report	YAML→Cypher 생성, 중복 ID 검사	재실행해도 동일 결과, 중복 없음
AI 에이전트 연결	3~5일	개발	MCP client config, query examples	자연어→Cypher 시험, failure case 수집	핵심 질문 90% 이상 정확히 조회
협업·변경 승인	3~5일	프로젝트 오너	PR template, CODEOWNERS, CI	변경 제안 파일 자동 생성	승인 없는 그래프 변경 불가
유지보수 전환	1주	오너 + steward	운영 매뉴얼, freshness dashboard, restore test	stale entity 후보 생성	백업/재구축 테스트 성공, steward 확인

DB 스키마 자동 추출 구조

처음부터 모든 DB 엔진에 범용 connector framework를 만들 필요는 없습니다.

예를 들어:

```
extractors/
  mysql.py
  postgres.py

schemas/
  reservation/
    2026-10-25.yaml
  settlement/
    2026-10-25.yaml

normalizer/
  schema_to_graph.py
```

로 시작합니다.

표준화된 중간 포맷은 다음처럼 잡습니다.

```
database:
 uid: db:reservation
 type: mysql

tables:
 - uid: table:reservation.booking
  schema: reservation
  name: booking
  comment: 예약 메타데이터 설명

  columns:
   - uid: column:reservation.booking.id
    name: id
    type: bigint
    nullable: false

  primaryKey:
   - id

  foreignKeys:
   - name: fk_booking_supplier
    column: supplier_id
    targetTable: partner.partner
    targetColumn: id
```

이 YAML이 DB 종류 차이를 흡수하면 Neo4j를 나중에 Jena나 다른 저장소로 바꾸더라도 원천 데이터를 다시 만들 필요가 없습니다.

Git을 중심으로 한 협업 절차

Git이 원본, 그래프가 배포 산출물이라는 원칙을 권장합니다.

```
팀원 자연어 요청
      ↓
AI agent
      ↓
ChangeProposal YAML
      ↓
feature branch / PR
      ↓
자동 검증
   ├─ ID 중복?
   ├── 필수 field?
   ├── 존재하지 않는 target?
   ├── 금지된 개인정보 패턴?
   ├─ sourceRef 존재?
   └─ steward 지정?
```

```
      ↓
Data Steward review
      ↓
필요 시 Tech Owner review
      ↓
merge
      ↓
CI service account
      ↓
Neo4j 갱신
      ↓
post-deploy competency query
```

PR 예시는 다음 정도면 충분합니다.

```
change:
 type: update

entity:
 uid: step:booking-cancellation:verify

before:
 description: "취소 조건 확인"

after:
 description: "예약 상태와 공급사 정책을 확인하여 취소 가능 여부를 판정"

reason: "2026-10 CS 프로세스 워크숍 결과"

sourceRefs:
 - source:workshop:cs-cancellation-2026-10

proposedBy: user:abc
reviewers:
 - role:cs-process-steward
```

AI 에이전트에게 직접 `CREATE`, `DELETE`, `SET` 권한을 주는 것보다 훨씬 안전합니다.

데이터 스튜어드 역할

데이터 스튜어드(Data Steward)는 개발자가 아니라 “그 정보가 지금도 사실인지 책임지는 현업 담당자”입니다.

역할	책임
Ontology Owner	클래스·관계·명명 규칙 승인
System/Technical Steward	시스템·API·DB 구조 확인

역할	책임
Department Steward	부서·역할·업무 용어 확인
Process Steward	프로세스 단계와 책임·예외 확인
Security Owner	공개 범위·민감도·MCP 정책
Platform Maintainer	Neo4j, extractor, CI, backup 운영

예를 들어 “booking 테이블에 FK가 생겼다”는 기술 steward가 확인하고, “환불 승인 책임자가 CS인가 정산인가”는 process steward가 확인합니다.

정보가 낡지 않게 만드는 방식

자동 freshness와 **사람 검증 freshness**를 분리합니다.

권고 주기는 다음과 같습니다. 이는 제품 제한이 아니라 **운영 목표값**입니다.

정보	권고 갱신
DB schema	매일 또는 배포 후 자동
API/integration	CI/CD 배포 시 + 월간 확인
system inventory	월간
process	90일마다 steward 재검증
owner/steward	90일마다
외부 공급사/PG 정보	월간 또는 계약 변경 시

각 객체에:

```
firstSeenAt
lastSeenAt
updatedAt
lastVerifiedAt
validFrom
validTo
status
```

를 두는 것이 좋습니다.

자동 스캔에서 사라진 테이블은 즉시 삭제하지 말고:

```
active
  ↓
missing-from-source
  ↓
```

```
deprecated
  ↓
retired
```

처럼 상태 전이시키면 “예전에는 있었던 테이블”까지 추적할 수 있습니다.

성공 측정·실패 예방·확장 기준

역량 질문 스무 개

역량 질문(Competency Question)은 “그래프가 이 질문에 정확히 답할 수 있으면 모델이 충분히 유용하다”고 정의하는 테스트 케이스입니다. 온톨로지를 클래스 수가 아니라 실제 업무 질문으로 검증하는 방법입니다.

구분	역량 질문
영향도	**예약 테이블을 변경하면 영향받는 시스템·프로세스·부서는 무엇인가?**
영향도	`booking.status` 컬럼을 사용하는 업무 단계는 어디인가?
영향도	특정 외부 PG가 중단되면 영향받는 시스템과 프로세스는 무엇인가?
영향도	특정 공급사 API를 호출하는 모든 시스템·모듈은 무엇인가?
영향도	한 시스템을 폐기하려면 먼저 대체해야 할 dependency는 무엇인가?
데이터	예약→결제→환불→정산 테이블 사이 FK 또는 논리적 연결 경로는 무엇인가?
데이터	특정 테이블을 읽는 프로세스와 쓰는 프로세스를 각각 보여 달라.
데이터	정산 프로세스가 쓰는 모든 DB·테이블·컬럼은 무엇인가?
데이터	시스템에는 연결되어 있지만 어떤 업무 프로세스에도 연결되지 않은 테이블은 무엇인가?
데이터	설명·owner·source가 빠진 테이블은 무엇인가?
프로세스	예약 취소의 전체 단계, 순서, 담당 역할을 보여 달라.
프로세스	환불 승인 단계에서 어떤 시스템과 테이블을 사용하는가?
프로세스	CS가 처리한 취소가 정산에 반영될 때까지 어떤 단계를 거치는가?
프로세스	수동 업무 단계는 무엇이며 자동화된 단계는 무엇인가?
프로세스	담당 역할이 정의되지 않은 process step은 무엇인가?
거버넌스	90일 이상 검증되지 않은 지식은 무엇인가?
거버넌스	특정 관계의 근거 문서 또는 DB introspection source는 무엇인가?
거버넌스	사람 검증 없이 자동 생성 상태로 남아 있는 객체는 무엇인가?
보안	일반 사용자 MCP가 수정할 수 있는 객체가 있는가? 정답은 원칙적으로 없어야 한다.
운영	지난 한 달간 어떤 schema change가 그래프에 반영됐고 누가 승인했는가?

측정 방법

파일럿 시작 전에 각 질문에 대해 전문가가 **정답지(gold set)**를 만듭니다.

그다음 AI 에이전트 응답을 다음 기준으로 평가합니다.

지표	파일럿 권고 목표
20개 역량 질문 정확도	**90% 이상**
근거 `sourceRef`가 있는 답변	**95% 이상**
파일럿 시스템·DB inventory coverage	**100%**
source에서 발견된 대상 테이블/컬럼 수집률	**100% 또는 승인된 제외목록**
파일럿 process step의 담당 Role 연결률	**100%**
각 step의 System 연결률	**100%**
각 데이터 접근 step의 READ/WRITE table 연결률	**100%**
필수 governance field 완성도	**95% 이상**
read-only MCP에서 write 성공 건수	**0**
승인 없는 production graph write	**0**
실고객/실결제 row 적재 건수	**0**
disaster recovery 재구축 테스트	**성공**

이 숫자는 벤더 기준이 아니라 **줌줌투어 내부 acceptance target 권고값**입니다.

AI 답변은 “문장이 그럴듯한가”로 평가해서는 안 됩니다.

예:

```
질문
"booking 변경 시 CS가 영향받는가?"

좋지 않은 답
"네, CS가 영향을 받을 가능성이 있습니다."

좋은 답
"예. booking은 예약취소 프로세스의
'취소 가능 여부 확인' 단계가 READS_FROM하며,
해당 단계의 PERFORMED_BY 역할은 CS Agent이고
그 역할은 CS 부서 소속입니다.
근거: source:..."
```

처럼 **그래프 경로 + 출처**가 있어야 합니다.

흔한 실패 요인과 예방책

실패	왜 실패하는가	예방
처음부터 회사 전체를 모델링	끝없이 클래스만 늘어남	프로세스 하나로 시작
600개 테이블을 먼저 다 적재	“그래서 뭘 물어볼 수 있지?”가 됨	역량 질문을 먼저 작성
데이터 카탈로그와 ontology를 동일시	테이블 목록은 있지만 업무 의미가 없음	BusinessEntity·Process·Role을 별도 모델링
AI 자동 추출을 사실로 간주	hallucination이 지식이 됨	`confidence:auto` → 사람 검증 후 `verified`
AI에게 graph write 직접 허용	잘못된 DELETE/MERGE 가능	read-only MCP + Git PR
사람이 직접 Browser에서 수정	Git과 DB가 달라짐	production direct write 금지
ID에 display name 사용	명칭 변경 시 graph가 깨짐	immutable `uid`
provenance 없음	왜 맞는지 검증 불가	모든 핵심 객체 `sourceRef`
`updatedAt`만 사용	오래됐는지 알 수 없음	`lastVerifiedAt` 별도
ERD만 신뢰	업무상 논리 관계와 실제 사용이 빠짐	인터뷰·코드·DB를 교차검증
모든 컬럼을 business concept와 연결	유지비 폭증	업무 핵심 컬럼만 semantic mapping
처음부터 OWL reasoning	모델링 학습이 프로젝트 목적이 됨	v1은 property graph
Graphiti로 자동 지식 생성부터 시작	검증·권한 문제가 더 어려워짐	자동 추출은 후단 proposal에만 사용
UI 시각화에 집착	예쁜 그래프가 업무 가치와 동일하지 않음	competency query가 우선
steward가 없음	몇 달 후 틀린 정보가 됨	부서별 책임자 지정
메타데이터 계정이 row까지 읽음	불필요한 개인정보 리스크	metadata-only privilege

삼 개월 시점 확장 기준

아래는 **본 조사 권고 기준**입니다.

파일럿을 3개월 운영한 뒤 다음이 충족되면 두 번째·세 번째 프로세스로 확장합니다.

```
역량 질문 정확도 ≥ 90%
핵심 entity provenance ≥ 95%
필수 field 완성도 ≥ 95%
```

```
자동 schema ingestion 안정 운영
critical stale item < 10%
read-only 보안 사고 = 0
현업 steward가 실제 PR review 수행
팀에서 월간 반복 질의가 발생
```

확장 순서는 보통:

```
예약 취소/환불
    ↓
예약 생성/확정
    ↓
바우처 발급
    ↓
공급사 정산
    ↓
CS escalation
    ↓
상품 등록/변경
```

처럼 업무 가치와 시스템 교차도가 높은 순서가 좋습니다.

육 개월 시점 기술 확장 기준

6개월 후에는 “현재 경량 스택이 여전히 맞는가?”를 다시 판단합니다.

DataHub/OpenMetadata로 확대할 신호는:

```
DB 엔진 종류와 수가 계속 증가
수천 단위 table/column을 수동 connector로 유지하기 힘듦
사용자가 카탈로그 웹 UI를 강하게 요구
column-level lineage 자동 수집 요구
팀별 RBAC가 꼭 필요
dashboard/dbt/BI까지 연결해야 함
```

입니다.

특히 자동 카탈로그가 핵심 병목이면 DataHub/OpenMetadata가 직접 만든 extractor보다 경제적일 수 있습니다. OpenMetadata는 현재 MySQL/PostgreSQL을 포함한 데이터베이스 connector를 통해 schema metadata와 lineage 등의 기능을 제공합니다. [58]

RDF/Jena 쪽으로 확대할 신호는:

```
외부 기관과 ontology를 표준 형식으로 교환
여러 회사가 동일 vocab을 공유
```

```
OWL reasoning 필요
sameAs/subClassOf 같은 표준 semantic interop 중요
SHACL 기반 데이터 계약 필요
```

일 때입니다. Fuseki는 RDF/SPARQL 서버와 ACL, TDB2 live backup을 제공하므로 이 단계에서는 강한 후보가 됩니다. [59]

Neo4j Enterprise 또는 다른 권한형 플랫폼으로 옮겨야 하는 신호는:

```
부서마다 서로 다른 그래프 영역만 보여야 함
DB 레벨 세밀한 RBAC가 필수
24×7 무중단 HA가 필수
온라인 backup이 필수
cluster/read scale-out이 필수
감사상 shared credential이 허용되지 않음
```

일 때입니다. Community는 한 개의 표준 DB만 지원하고, 온라인 백업은 Enterprise 기능이며 고급 RBAC 역시 Enterprise 제품군의 영역입니다. [4]

즉 **무료판을 기술적으로 끝까지 억지로 쓰는 것이 목표가 아니라, 무료판으로 가치가 검증되기 전까지 불필요한 복잡성을 사지 않는 것**이 목표여야 합니다.

도입 전 직접 확인해야 할 체크리스트

현재 환경 정보가 미정이므로 실제 설치 전에 아래 질문을 채우는 것이 아키텍처 확정의 마지막 단계입니다.

구분	직접 확인할 항목	결정에 미치는 영향
규모	실제 직원 수 / 주간 활성 사용 예상 인원	MCP gateway·동시접속 설계
조직	실제 부서·팀 목록	ORG 모델·steward 구조
시스템	운영 시스템 전체 목록과 중요도	파일럿 scope·System 클래스
API	내부 API / 외부 PG / 공급사 API 수	Integration 모델
DB	MySQL/PostgreSQL/기타 종류와 버전	extractor 선택
DB 규모	DB·schema·table·column 개수	custom extractor vs DataHub/OpenMetadata
접근권한	`information_schema`만으로 필요한 metadata를 모두 볼 수 있는지	source account 설계
서버	실제 Ubuntu/CPU/RAM/SSD	Neo4j 단독 또는 catalog platform 가능 여부
Docker	Docker/Compose 사용 가능 여부	설치 방식
백업	야간 수분 downtime이 허용되는지	Neo4j CE offline dump 가능 여부

구분	직접 확인할 항목	결정에 미치는 영향
DR	Graph를 Git에서 재생성해도 되는 RTO/RPO인지	CE 지속 사용 판단
인증	Google Workspace SSO/VPN/IdP 여부	MCP gateway
네트워크	Claude/Cursor PC에서 내부 MCP에 도달하는 경로	VPN/Zero Trust
AI	Claude Code/Desktop/Cursor 등 실제 client	MCP transport 검증
LLM	클라우드 LLM 사용 허용 여부	데이터 분류·local LLM 필요성
계약	Claude/OpenAI 등 실제 요금제의 DPA·보존·학습 정책	외부 전송 허용 범위
문서	Notion/Confluence/ERD/BPMN/README의 실제 현황	AI-assisted bootstrap
코드	Git 저장소의 API/ORM/schema migration을 분석할 수 있는지	DB 외 근거 확보
담당자	실제 주당 투입시간	8~12주 일정 현실성
steward	예약·CS·정산·개발에서 각 1명 지정 가능한지	장기 freshness
개인정보	DB comment·SQL query text에 개인정보나 secret이 들어가는 사례가 있는지	ingestion filtering
Neo4j	도입 시점 최신 Community 버전과 GPLv3 정책 재확인	라이선스
Neo4j MCP	`NEO4J_READ_ONLY=true`와 사용하는 client 버전의 동작 실검증	안전한 MCP 운영
OpenMetadata	사용할 경우 core와 `openmetadata-mcp`의 라이선스가 여전히 분리되어 있는지	법무 검토 14
OpenMetadata	실제 서버가 공식 production 요구량을 감당하는지	제품 선택 40
DataHub	사용할 경우 quickstart가 아닌 production deployment 방식을 확정했는지	운영 안정성 34
WebProtégé	배포 시점 안정 self-host release와 유지되는 배포 저장소 확인	RDF 대안
Graphiti	reference MCP를 production에 직접 노출하지 않는지	인증·인가 위험 37
모델 라이선스	Ollama 위에서 사용할 로컬 LLM 자체 라이선스 확인	상업적 내부 이용
Git 정책	production graph를 사람이 직접 수정하지 못하게 할 수 있는지	거버넌스 핵심
ID 정책	변경되지 않는 canonical UID 규칙을 합의했는지	장기 유지보수
출처 정책	모든 수동 입력에 `sourceRef`를 강제할지	신뢰성
검증 정책	`lastVerifiedAt` 만료기간과 담당자를 정했는지	노후화 방지
파일럿	예약 취소·환불 같은 첫 프로세스 하나가 명확히 지정됐는지	범위 통제
성공 기준	파일럿 전 역량 질문과 정답지를 먼저 만들었는지	프로젝트 가치 측정

최종적으로 1차 목표를 한 문장으로 정의하면 다음과 같습니다.

“줌줌투어의 시스템·DB 스키마·외부 연동·부서·역할·업무 프로세스를 변경 가능한 코드와 출처가 있는 그래프로 연결하고, 직원의 AI 에이전트는 그 그래프를 읽기 전용으로 질의하며, 모든 수정은 Git 제안과 사람 승인을 거쳐 반영되게 한다.”

이 정의를 지키면 초기에는 Neo4j Community처럼 비교적 단순한 무료 저장소로도 충분히 큰 가치를 만들 수 있고, 향후 실제 병목이 **카탈로그 자동화**인지, **권한/HA**인지, **RDF 표준화**인지가 드러난 뒤 DataHub·OpenMetadata·Jena 또는 상위 에디션으로 확장할 수 있습니다.

---

1 9 Neo4j Licensing | Open Source Graph Databases & License Overview
https://neo4j.com/licensing/?ref=release-notes&utm_source=chatgpt.com

2 5 RDF 1.1 Concepts and Abstract Syntax
https://www.w3.org/TR/rdf11-concepts/?utm_source=chatgpt.com

3 40 41 44 Production-Ready Requirements for OpenMetadata Deployment - OpenMetadata Documentation
https://docs.open-metadata.org/v2.0.x/deployment/production-ready-requirements?utm_source=chatgpt.com

4 Database administration - Operations Manual
https://neo4j.com/docs/operations-manual/current/database-administration/?utm_source=chatgpt.com

6 OWL 2 Web Ontology Language Document Overview (Second Edition)
https://www.w3.org/TR/owl2-overview/?utm_source=chatgpt.com

7 8 https://neo4j.com/docs/operations-manual/2026.10/
https://neo4j.com/docs/operations-manual/2026.10/?utm_source=chatgpt.com

10 59 Apache Jena - Apache Jena Releases
https://jena.apache.org/download/?utm_source=chatgpt.com

11 Releases · protegeproject/protege-distribution
https://github.com/protegeproject/protege-distribution/releases?utm_source=chatgpt.com

12 GitHub - protegeproject/protege: Protege Desktop · GitHub
https://github.com/protegeproject/protege?utm_source=chatgpt.com

13 2.0 Release | OpenMetadata - OpenMetadata Documentation
https://docs.open-metadata.org/v2.0.x/releases/2.0-release?utm_source=chatgpt.com

14 OpenMetadata/LICENSE at main · open-metadata/OpenMetadata · GitHub
https://github.com/open-metadata/OpenMetadata/blob/main/LICENSE?utm_source=chatgpt.com

15 Releases · datahub-project/datahub
https://github.com/datahub-project/datahub/releases?utm_source=chatgpt.com

16 datahub/LICENSE at master · datahub-project/datahub · GitHub
https://github.com/datahub-project/datahub/blob/master/LICENSE?utm_source=chatgpt.com

17 46 Apache AGE Graph Database | Apache AGE
https://age.apache.org/release-notes/?utm_source=chatgpt.com

18 Releases · JanusGraph/janusgraph
https://github.com/JanusGraph/janusgraph/releases?utm_source=chatgpt.com

19 JanusGraph · Distributed, open source, massively scalable graph database
https://janusgraph.org/?utm_source=chatgpt.com

20 Releases · getzep/graphiti
https://github.com/getzep/graphiti/releases?utm_source=chatgpt.com

21 graphiti/LICENSE at main · getzep/graphiti · GitHub
https://github.com/getzep/graphiti/blob/main/LICENSE?utm_source=chatgpt.com

22 25 Protégé
https://protege.stanford.edu/?utm_source=chatgpt.com

23 Changes in Neo4j 2025-2026 series - Operations Manual
https://neo4j.com/docs/operations-manual/current/changes-2025-2026/?utm_source=chatgpt.com

24 45 Apache Jena - Data Access Control for Fuseki
https://jena.apache.org/documentation/fuseki2/fuseki-data-access-control?utm_source=chatgpt.com

26 Role-based access control - Operations Manual
https://neo4j.com/docs/operations-manual/current/authentication-authorization/manage-privileges/?utm_source=chatgpt.com

27 Configuration reference - Neo4j MCP
https://neo4j.com/docs/mcp/current/configuration/?utm_source=chatgpt.com

28 Apache Jena - Fuseki : Plain Server
https://jena.apache.org/documentation//fuseki2/fuseki-plain?utm_source=chatgpt.com

29 43 Backup and restore - Operations Manual
https://neo4j.com/docs/operations-manual/current/backup-restore/?utm_source=chatgpt.com

30 Apache Jena - TDB2 - Database Administration
https://jena.apache.org/documentation/tdb2/tdb2_admin.html?utm_source=chatgpt.com

31 47 MySQL Connector | OpenMetadata Database Integration Guide - OpenMetadata Documentation
https://docs.open-metadata.org/v2.0.x/connectors/database/mysql?utm_source=chatgpt.com

32 DataHub MCP Server | DataHub
https://docs.datahub.com/docs/features/feature-guides/mcp?utm_source=chatgpt.com

33 39 Graphiti MCP Server | Zep Documentation
https://help.getzep.com/graphiti/getting-started/mcp-server?utm_source=chatgpt.com

34 DataHub Quickstart Guide | DataHub
https://docs.datahub.com/docs/quickstart?utm_source=chatgpt.com

35 Apache AGE Graph Database | Apache AGE
https://age.apache.org/getstarted/quickstart/?utm_source=chatgpt.com

36 JanusGraph Container - JanusGraph
https://docs.janusgraph.org/v1.1/operations/container/?utm_source=chatgpt.com

37 GitHub - getzep/graphiti: Build Real-Time Knowledge Graphs for AI Agents · GitHub
https://github.com/getzep/graphiti?utm_source=chatgpt.com

38 OpenMetadata/openmetadata-mcp/LICENSE at main · open-metadata/OpenMetadata · GitHub
https://github.com/open-metadata/OpenMetadata/blob/main/openmetadata-mcp/LICENSE

42 Client configuration - Neo4j MCP
https://neo4j.com/docs/mcp/current/client-configuration/?utm_source=chatgpt.com

48 Authorization - Model Context Protocol
https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization?utm_source=chatgpt.com

49 ollama/LICENSE at main · ollama/ollama · GitHub
https://github.com/ollama/ollama/blob/master/LICENSE?utm_source=chatgpt.com

50 The Organization Ontology
https://www.w3.org/TR/vocab-org/?utm_source=chatgpt.com

51 PROV-O: The PROV Ontology
https://www.w3.org/TR/prov-o/?utm_source=chatgpt.com

52 Data Catalog Vocabulary (DCAT) - Version 3
https://www.w3.org/TR/vocab-dcat-3/?utm_source=chatgpt.com

53 SKOS Simple Knowledge Organization System Reference
https://www.w3.org/TR/skos-reference/?utm_source=chatgpt.com

54 DCMI: DCMI Metadata Terms
https://www.dublincore.org/specifications/dublin-core/dcmi-terms/default.htm?utm_source=chatgpt.com

55 Organization - Schema.org Type
https://schema.org/Organization?utm_source=chatgpt.com

56 Business Process Model and Notation (BPMN), Version 2
https://www.omg.org/spec/BPMN/2.0.2/PDF/?utm_source=chatgpt.com

57 Shapes Constraint Language (SHACL)
https://www.w3.org/TR/shacl/?utm_source=chatgpt.com

58 PostgreSQL Connector | OpenMetadata Database Integration - OpenMetadata Documentation
https://docs.open-metadata.org/v2.0.x/connectors/database/postgres?utm_source=chatgpt.com